

■ COMPUTER NETWORKS

2nd MID EXAM — COMPLETE SOLUTION GUIDE

Application Layer • HTTP • DNS • Email • Cookies • Web Caching • TCP/UDP

Short Answer Questions ✓	MCQs Solved ✓	Simple Explanations	Full Question Before Each Ans
8	22		

■ HOW TO USE THIS GUIDE

Each short-answer question shows the FULL question statement first, then a clear simple answer.

Each MCQ shows all options — the correct answer is in GREEN with ✓. Then a simple explanation.

Everything is explained like you are new to networking. Real-life examples used throughout! ■

TABLE OF CONTENTS

SECTION	CONTENT	TOPIC
Part 1	Short Answer Questions (Q2, Q6–Q11 + Unnumbered)	Delays, UDP/TCP, TLS, Cookies, Web Cache, Email, Packet Switching
Part 2	Multiple Choice Questions (MCQ 1–22)	Application Layer, HTTP, DNS, Email, Web Protocols



PART 1: SHORT ANSWER / DESCRIPTIVE QUESTIONS

TIP

Read the FULL question (grey box), then read the answer. Try to understand the concept, not just memorize words. Each answer has a real-life example to make it easy! ■

Question 2 — End-to-End Delay Components

QUESTION:

Consider sending a packet from a source host to a destination host over a fixed route. List the delay components in the end-to-end delay. Which of these delays are constant and which are variable?

What is End-to-End Delay?

When you send a packet from your computer to another computer far away, the packet doesn't arrive instantly. It experiences several types of delays along the way. Think of it like sending a physical package through multiple warehouses to reach its destination.

The 4 Delay Components:

Delay Type	What Happens	Real-Life Example	Constant or Variable?
1. Processing Delay	Router reads packet header and decides where to forward it	Cashier scanning your item at checkout	■ Constant (microseconds, fixed)
2. Queuing Delay	Packet waits in a buffer/queue at the router before being sent	Waiting in a long queue at a bank	■ Variable (depends on traffic — can be 0 or very high)
3. Transmission Delay	Time to push ALL bits of the packet onto the wire = $\text{Packet size} \div \text{Link bandwidth}$	Time to print all pages of a book on a printer	■ Constant (fixed: packet size and link speed don't change)
4. Propagation Delay	Time for one bit to travel from sender to receiver = $\text{Distance} \div \text{Speed of light}$	How long sound takes to travel across a big hall	■ Constant (fixed: distance and medium don't change on fixed route)

■ Summary: Only QUEUING DELAY is variable — it changes based on how busy the network is. The other three (processing, transmission, propagation) are essentially CONSTANT for a fixed route because the route, distances, packet sizes, and link speeds are all fixed.

Question 6 — UDP vs TCP for Fast Transactions

QUESTION:

Suppose you wanted to do a transaction from a remote client to a server as fast as possible. Would you use UDP or TCP? Why?

■ Answer: It Depends on What 'Transaction' Means — But Generally UDP for Raw Speed

This is a nuanced question. Let's understand both protocols first:

Feature	TCP	UDP
Connection	3-way handshake required BEFORE data (adds delay)	No handshake — just send immediately ■
Reliability	Guarantees delivery, ordering, error correction	No guarantee — fire and forget
Speed	Slower due to handshake + acknowledgements	Faster — minimal overhead
Overhead	High — lots of control messages	Low — simple header
Use Case	File transfer, email, web browsing	Video streaming, gaming, DNS, VoIP

For a **single short transaction** (like a DNS lookup or a quick ping), **UDP is faster** because TCP needs a 3-way handshake (SYN → SYN-ACK → ACK) before even one byte of data is sent. That's 1.5 RTTs of delay just to start!

However, if the transaction requires **reliability** (e.g., a bank transaction, login credentials), **TCP is the correct choice** because losing or corrupting data would be catastrophic. Speed without correctness is useless for transactions.

■ Final Answer: Use UDP if the transaction is small and speed is priority (e.g., DNS). Use TCP if the transaction needs reliability (e.g., banking, login). For truly fast raw transactions: UDP wins because it skips the connection setup entirely.

Question 7 — TLS: Transport Layer or Application Layer?

■ QUESTION:

Recall that TCP can be enhanced with TLS to provide process-to-process security services, including encryption. Does TLS operate at the transport layer or the application layer? If the application developer wants TCP to be enhanced with TLS, what does the developer have to do?

■ What is TLS?

TLS = Transport Layer Security. Despite the name including 'Transport', TLS actually operates at the **Application Layer**. Let's understand why:

Think of TLS like a secure safe inside a delivery truck. The truck (TCP) handles the delivery. The safe (TLS) handles security. The safe is loaded and opened by the APPLICATION (the sender and receiver programs) — not by the delivery truck itself.

■ Where Does TLS Operate?

TLS is implemented in the **Application Layer**. Even though it sits between the application and TCP (transport), it is handled as application-layer code. TLS is NOT built into TCP itself — it is a library/API that applications call.

■■■ What Must the Developer Do?

Step 1

Include a TLS library in the application code (e.g., OpenSSL, BoringSSL, Java's SSLSocket).

Step 2	Instead of using regular TCP sockets, use TLS sockets provided by the library.
Step 3	The TLS library handles: encryption, certificate verification, and key exchange automatically.
Step 4	The app sends data through TLS socket → TLS encrypts it → TCP transmits the encrypted bytes.
Step 5	On the receiving end, TLS socket decrypts the data before the application sees it.

■ TLS operates at the APPLICATION LAYER (even though the name says 'Transport'). To use TLS, the developer replaces standard TCP socket calls with TLS socket calls using a TLS library. The rest (encryption, handshake, certificates) is handled by the library.

Question 8 — Cookies: Keeping Purchase Records on E-Commerce Sites

■ QUESTION:

Consider an e-commerce site that wants to keep a purchase record for each of its customers. Describe how this can be done with cookies.

■ What Are Cookies?

HTTP is a STATELESS protocol — every request is independent and the server has NO memory of who you are between requests. Cookies solve this by storing a small ID on YOUR computer that is sent to the server with every request.

Think of cookies like a **loyalty card** at a shop. Every time you visit, you show your card. The shop's system looks up your card number and finds all your history. The card itself (cookie) only holds an ID — the real data is in the shop's database (server).

■ How an E-Commerce Site Uses Cookies for Purchase Records:

Visit #1 (New User)	Server creates a unique ID (e.g., user_id=7823) and stores it in its database. Server sends: Set-Cookie: user_id=7823 in the HTTP response header.
Browser Stores It	Browser saves the cookie: user_id=7823 on your computer.
Next Request	Every time you visit the site, browser automatically sends Cookie: user_id=7823 in the request header.
Server Looks You Up	Server reads user_id=7823, looks in database → finds your name, cart items, past purchases, preferences.
Purchase Made	When you buy something, server records: user_id=7823 bought 'laptop' on April 6, 2026 — stored in database.
Future Visits	Next visit: server sees your ID, loads your purchase history, shows recommendations, auto-fills your details.

■ Cookies work in 4 parts: (1) Cookie header line in HTTP response — server sets the cookie. (2) Cookie file in browser — browser stores it. (3) Cookie header line in HTTP request — browser sends it back. (4) Back-end database on server — where all actual user data is stored.

Question 9 — Web Caching: Reducing Delay

■ QUESTION:

Describe how Web caching can reduce the delay in receiving a requested object. Will Web caching reduce delay for all objects requested by a user or for only some of the objects? Why?

■ ■ What is Web Caching (Proxy Server)?

A Web Cache (also called a proxy server) is a computer that stores COPIES of recently requested web objects (images, HTML files, videos). It sits between users and the internet. Think of it as a **local library** — instead of ordering a book from a faraway publisher (original server), you check the local library first (cache).

■ How It Reduces Delay:

Without cache: Browser → Internet → Origin Server (far away, slow, congested link) → back

With cache: Browser → Local Cache (nearby, fast LAN) → object returned instantly!

The cache checks: 'Do I have a fresh copy of this object?' If YES → serve immediately. If NO → fetch from origin server, store a copy, then serve to user.

■ How Much Does It Help?

Scenario	Without Cache	With Cache (Hit)
Request goes to:	Origin server (possibly far, slow)	Local cache server (fast LAN)
Access link usage:	High bandwidth consumption	No access link used for cached items
Response time:	Propagation + queuing + transmission delay	Milliseconds (LAN speed)
Server load:	Origin server handles every request	Origin server only handles cache misses

■ Does Caching Reduce Delay for ALL Objects?

NO — only for SOME objects. Caching only helps when there is a 'cache hit' (the requested object is already stored in the cache). Delay is NOT reduced when:

- The object is being requested for the **first time ever** (cache miss).
- The cached copy has **expired** (TTL elapsed) and a fresh copy must be fetched.
- The object is **dynamic** or personalized (e.g., your bank balance page — can't be cached for everyone).
- The object is very large and the cache didn't store it to save space.

■ Web caching reduces delay ONLY for cached objects (cache hits). First-time requests, expired content, and dynamic/personalized content still go to the origin server and experience full delay. In practice, a hit rate of 40–60% is typical, significantly reducing average delay and bandwidth use.

Question 10 — Alice Sends Email to Bob: Full Message Journey

■ QUESTION:

Suppose Alice, with a Web-based e-mail account (such as Hotmail or Gmail), sends a message to Bob, who accesses his mail from his mail server using IMAP. Discuss how the message gets from Alice's host to Bob's host. Be sure to list the series of application-layer protocols that are used to move the message between the two hosts.

■ The Full Email Journey (Step by Step)

This is a 3-hop journey. Alice uses a WEB browser (no email client), Bob uses IMAP (email client like Outlook/Thunderbird). Here's what happens:

Step 1 Alice → Gmail Server

Alice types her email in Gmail's web interface (browser). She clicks Send. Protocol used: HTTP (or HTTPS) — because she's using a web browser, the message goes from her browser to Gmail's web server over HTTP. This is NOT SMTP yet!

Step 2 Gmail Server → Bob's Mail Server

Gmail's mail server now forwards the email to Bob's mail server. Protocol used: SMTP (Simple Mail Transfer Protocol) — this is always used for SERVER-TO-SERVER email transfer. Gmail's SMTP server contacts Bob's domain's SMTP server and delivers the message. Port 25 is used.

Step 3 Bob Downloads His Mail

Bob opens his email client (Outlook, Thunderbird, etc.) and it connects to his mail server. Protocol used: IMAP (Internet Mail Access Protocol) — Bob's client uses IMAP to retrieve and manage mail stored on the server. (POP3 could also be used, but IMAP is specified here.) IMAP keeps mail on the server; POP3 downloads and deletes.

Protocol	Used Between	Why This Protocol?
HTTP / HTTPS	Alice's browser → Gmail server	Alice uses a web interface, so HTTP is used to submit the email form
SMTP	Gmail's mail server → Bob's mail server	Standard protocol for mail transfer between servers (port 25)
IMAP	Bob's mail client → Bob's mail server	Bob retrieves mail from server; IMAP keeps mail on server for multi-device access

■ Protocol chain: HTTP (Alice → Gmail) → SMTP (Gmail → Bob's server) → IMAP (Bob's client → server). Note: If Alice used a desktop email client (Outlook), Step 1 would also be SMTP. HTTP is only used because Alice is using a web-based email service.

Question 11 — Circuit Switching vs Packet Switching (2 Mbps Link)

■ QUESTION:

Suppose users share a 2 Mbps link. Also suppose each user transmits continuously at 1 Mbps when transmitting, but each user transmits only 20 percent of the time. A. When circuit switching is used, how many users can be supported? B. Why will there be essentially no queuing delay before the link if two or fewer users transmit at the same time? Why will there be queuing delay if three users transmit at the same time? C. Find the probability that a given user is transmitting.

■ Given Information:

- Total link capacity = **2 Mbps**
- Each user's transmission rate = **1 Mbps** (when active)
- Each user is active only **20% of the time** (probability = 0.2)

Part A: Circuit Switching — How Many Users?

In circuit switching, each active user gets a **DEDICATED** slice of bandwidth. Since each user needs 1 Mbps and the total link is 2 Mbps:

Max users = Total bandwidth ÷ Per-user bandwidth = **2 Mbps ÷ 1 Mbps = 2 users**

Even if only 1 user is actively sending, the other slot is **reserved and wasted**. This is the big inefficiency of circuit switching!

■ Part A Answer: Circuit switching supports only **2 users** on this 2 Mbps link.

Part B: Packet Switching — Queuing Delay Analysis

Why NO queuing delay with ≤ 2 users transmitting simultaneously?

If only 1 or 2 users are transmitting at the same time, their combined data rate is at most $1+1 = 2$ Mbps, which exactly equals the link capacity. The link can handle everything instantly — no backlog, no queue!

Why IS there queuing delay with 3 users transmitting simultaneously?

If 3 users transmit simultaneously, total demand = 3×1 Mbps = **3 Mbps**, but the link can only carry 2 Mbps. The extra 1 Mbps of data has nowhere to go — it must **wait in a buffer (queue)**. This waiting time is queuing delay!

■ Part B: With ≤ 2 users: demand $\leq$ capacity (2 Mbps) → no queue. With 3 users: demand (3 Mbps) $>$ capacity (2 Mbps) → overflow goes into queue → queuing delay!

Part C: Probability That a Given User is Transmitting

This is directly given in the problem! Each user is active 20% of the time.

Probability that a given user is transmitting at any moment = **p = 0.20 (20%)**

■ Part C: $P(\text{user is transmitting}) = 0.20 = 20\%$. This means at any random moment, there is a 1 in 5 chance that a given user is sending data.

Unnumbered Question — Packets & Router Forwarding

■ QUESTION:

Suppose end system A wants to send a large file to end system B. At a very high level, describe how system A creates packets from the file. When one of these packets arrives to a router, what information in the packet does the router use to determine the link onto which the packet is forwarded?

■ Part 1: How System A Creates Packets From a File

Sending a large file directly is impossible in one chunk — the network handles small pieces. Here's what happens:

Segmentation	The application (e.g., FTP) passes the file to the Transport layer (TCP). TCP breaks the file into small chunks called SEGMENTS. Maximum size is determined by MTU (Maximum Transmission Unit) — typically ~1500 bytes for Ethernet.
Headers Added	Each segment gets headers added at each layer: Transport layer adds TCP header (source/dest port, sequence number). Network layer adds IP header (source/dest IP address). This wrapped unit is now called a PACKET.
Sequence Numbers	TCP numbers each segment so that even if they arrive out of order, the receiver can reassemble the original file correctly.
Transmission	Packets are sent one by one (or in parallel with pipelining) onto the network.

■ Part 2: What Does the Router Use to Forward the Packet?

When a packet arrives at a router, the router does NOT look at the application data. It looks at the **Destination IP Address** in the IP header (Network Layer).

The router has a **Forwarding Table** (routing table). It matches the destination IP against entries in this table using **longest-prefix matching** to find the best outgoing link.

Example: Destination IP = 192.168.5.23 → Router checks table → matches entry for 192.168.5.0/24 → forwards packet out of interface eth2.

■ Summary: System A segments the file → adds headers (TCP + IP) → creates packets. Router uses the DESTINATION IP ADDRESS in the packet's IP header to look up its forwarding table and decide which outgoing link to use. MAC addresses are used only within local networks (not by routers for routing).

■ PART 2: MULTIPLE CHOICE QUESTIONS (MCQ 1–22)

■ TIP

Each MCQ shows all options. The GREEN ✓ option is correct. A simple explanation follows. Great for last-minute revision! ■

MCQ 1: Which is NOT an Application Layer protocol?

A. HTTP

B. SMTP

C. FTP

✓ D. TCP

■ Answer: D. TCP

■ *TCP (Transmission Control Protocol) lives in the TRANSPORT layer, not Application layer. HTTP (web), SMTP (email), and FTP (file transfer) are all Application layer protocols. TCP is the engine underneath them — it handles reliable delivery.*

MCQ 2: The packet of information at the Application layer is called _____.

A. Packet

✓ B. Message

C. Segment

D. Frame

■ Answer: B. Message

■ *Each layer has a different name for its data unit: Application = Message, Transport = Segment (TCP) or Datagram (UDP), Network/Internet = Packet, Data Link = Frame, Physical = Bits. At the Application layer, data is simply called a 'Message'.*

MCQ 3: Which one of the following is an architecture paradigm?

A. Peer to peer

B. Client-server

C. HTTP

✓ D. Both Peer-to-Peer & Client-Server

■ Answer: D. Both Peer-to-Peer & Client-Server

■ *There are two main network architecture models: Client-Server (like websites — one server serves many clients) and Peer-to-Peer (like BitTorrent — devices talk directly to each other). HTTP is a protocol, not an architecture.*

MCQ 4: Application layer offers _____ service.

A. End to end

✓ **B. Process to process**

C. Both

D. None

■ **Answer: B. Process to process**

■ *The Application layer provides communication between PROCESSES (running programs) on different hosts. 'End-to-end' is more of a Transport layer concept. The app layer specifically enables one application process (e.g., your browser) to communicate with another (e.g., a web server process).*

MCQ 5: E-mail is _____.

A. Loss-tolerant application

B. Bandwidth-sensitive application

✓ **C. Elastic application**

D. None

■ **Answer: C. Elastic application**

■ *An elastic application can adjust its performance based on available bandwidth. Email doesn't care if it arrives in 1 second or 5 seconds — no strict timing. It's NOT loss-tolerant (losing an email is bad!), NOT bandwidth-sensitive (doesn't need a fixed rate). So it's elastic.*

MCQ 6: Pick the odd one out.

A. File transfer

B. File download

C. E-mail

✓ **D. Interactive games**

■ **Answer: D. Interactive games**

■ *HTTP is used for: File transfer, file download, and email — all are classic Application layer tasks that are non-real-time or at least not interactive. Interactive games require real-time two-way communication and low latency, making them fundamentally different from the others.*

MCQ 7: Which of the following is an Application layer service?

- A. Network virtual terminal
- B. File transfer, access, and management
- C. Mail service

✓ **D. All of the mentioned**

■ **Answer: D. All of the mentioned**

■ *All three are Application layer services: Network virtual terminal (Telnet — remote login), File transfer, access and management (FTP), and Mail service (SMTP/IMAP/POP3). The Application layer provides all these user-facing services.*

MCQ 8: To deliver a message to the correct application running on a host, the _____ address must be consulted.

- A. IP
- B. MAC
- ✓ **C. Port**
- D. None

■ **Answer: C. Port**

■ *An IP address identifies the DEVICE (computer/phone). But one device runs many apps simultaneously. PORT numbers identify which APP on that device should receive the data. For example: port 80 = web browser, port 25 = email server, port 22 = SSH.*

MCQ 9: Which is a time-sensitive service?

- A. File transfer
- B. File download
- C. E-mail

✓ **D. Internet telephony**

■ **Answer: D. Internet telephony**

■ *Internet telephony (VoIP — like WhatsApp calls, Zoom) is extremely time-sensitive. A 1-second delay in a phone call is very noticeable and annoying. File transfer, download, and email can wait seconds or minutes — nobody complains!*

MCQ 10: Transport services available to applications include _____.

- A. Reliable data transfer
- B. Timing
- C. Security
- ✓ **D. All of the mentioned**

■ **Answer: D. All of the mentioned**

■ *Transport layer provides: Reliable data transfer (TCP guarantees delivery), Timing (some protocols guarantee delay bounds), and Security (TLS on top of TCP). All three are services the Transport layer can offer to applications. So 'All of the mentioned' is correct.*

MCQ 11: The _____ translates internet domain and host names to IP addresses.

- ✓ **A. Domain Name System**
- B. Routing Information Protocol
- C. Network Time Protocol
- D. Internet Relay Chat

■ **Answer: A. Domain Name System (DNS)**

■ *DNS is the internet's phonebook. When you type 'google.com', your computer asks DNS: 'What is the IP address for google.com?' DNS replies: '142.250.80.46'. RIP is a routing protocol, NTP is for time synchronization, IRC is for chat.*

MCQ 12: Which allows a user at one site to establish a connection to another site and pass keystrokes?

- A. HTTP
- B. FTP
- ✓ **C. Telnet**
- D. TCP

■ **Answer: C. Telnet**

■ *Telnet is a remote login protocol. You connect to a remote computer and everything you type is sent to and executed on the remote machine. It's like sitting in front of that computer. (Note: Telnet is insecure — SSH is the modern secure replacement.)*

MCQ 13: Application layer protocol defines _____.

- A. Types of messages exchanged
- B. Message format, syntax and semantics
- C. Rules for when and how processes send/respond

✓ **D. All of the mentioned**

■ **Answer: D. All of the mentioned**

■ *An application-layer protocol defines everything about how apps communicate: types of messages (request/response), message format/syntax/semantics (what each field means), and rules for when/how to send and respond. All three together define a complete protocol.*

MCQ 14: Which protocol delivers/stores mail to receiver's server?

✓ **A. Simple Mail Transfer Protocol**

- B. Post Office Protocol
- C. Internet Mail Access Protocol
- D. Hypertext Transfer Protocol

■ **Answer: A. Simple Mail Transfer Protocol (SMTP)**

■ *SMTP is used to SEND (push) email from sender to receiver's mail server. POP3 and IMAP are used to RECEIVE (pull) email from your mail server to your device. HTTP is for web pages. So SMTP is the one that 'delivers' to the receiver's server.*

MCQ 15: When displaying a web page, the application layer uses the _____.

✓ **A. HTTP protocol**

- B. FTP protocol
- C. SMTP protocol
- D. TCP protocol

■ **Answer: A. HTTP protocol**

■ *HTTP (HyperText Transfer Protocol) is the Application layer protocol for the World Wide Web. When your browser loads a web page, it sends an HTTP GET request and receives an HTTP response. FTP is for files, SMTP is for email, TCP is a Transport layer protocol.*

MCQ 16: A Web page with 4 JPEG images and 1 HTML text has _____ objects.

A. 4

B. 1

✓ C. 5

D. 7

■ **Answer: C. 5**

■ *In HTTP, every individual file is called an 'object'. Count them: 4 JPEG images + 1 HTML file = 5 total objects. The browser needs to send 5 separate HTTP requests to get all objects (in non-persistent HTTP). The HTML file is also 1 object, not just a container.*

MCQ 17: The default connection type used by HTTP is _____.

✓ A. Persistent

B. Non-persistent

C. Can be either depending on request

D. None

■ **Answer: A. Persistent**

■ *HTTP/1.1 (the most widely used version) uses PERSISTENT connections by default. This means the TCP connection stays open after a response so multiple objects can be fetched on the same connection. Non-persistent (HTTP/1.0) opened a new connection each time.*

MCQ 18: The HTTP request message is sent in _____ part of the three-way handshake.

A. First

B. Second

✓ C. Third

D. Fourth

■ **Answer: C. Third**

■ *TCP three-way handshake: (1) SYN — client to server, (2) SYN-ACK — server to client, (3) ACK — client to server. The HTTP GET request is piggybacked onto the THIRD message (ACK). This is why fetching a base HTML page takes 2 RTTs in non-persistent HTTP.*

MCQ 19: In fetching a web page, the HTTP request/response takes _____ RTTs.

A. 2

✓ **B. 1**

C. 4

D. 3

■ **Answer: B. 1**

■ After the TCP connection is established (which takes 1 RTT for the handshake), the actual HTTP request/response (GET → HTML file) takes exactly **1 more RTT**. So total for a single object in non-persistent HTTP = 2 RTTs. But the question asks only about the HTTP request/response phase = 1 RTT.

MCQ 20: The first line of an HTTP request message is called _____.

✓ **A. Request line**

B. Header line

C. Status line

D. Entity line

■ **Answer: A. Request line**

■ An HTTP request has 3 parts: (1) Request line — method, URL, HTTP version (e.g., GET /index.html HTTP/1.1), (2) Header lines — extra metadata (Host, Connection, etc.), (3) Entity body — data (used in POST). The Status line is the first line of a RESPONSE (e.g., HTTP/1.1 200 OK).

MCQ 21: The values GET, POST, HEAD etc. are specified in _____ of HTTP message.

✓ **A. Request line**

B. Header line

C. Status line

D. Entity body

■ **Answer: A. Request line**

■ The HTTP METHOD (GET, POST, HEAD, PUT, DELETE) is always in the FIRST line of the request message — the Request line. Format: [METHOD] [URL] [HTTP Version]. For example: GET /home.html HTTP/1.1

MCQ 22: The HTTP response message leaves out the requested object when _____ method is used.

A. GET

B. POST

✓ **C. HEAD**

D. PUT

■ **Answer: C. HEAD**

■ *The HEAD method asks the server: 'Give me ONLY the headers for this object, not the actual content.' It's used to check if a file exists, get its size, or check when it was last modified — without wasting bandwidth downloading the actual file. GET returns headers + body.*

■ This guide covers all 8 short-answer questions and all 22 MCQs from the 2nd Mid Exam.
Topics: Application Layer, TCP vs UDP, TLS, HTTP, Cookies, Web Caching, Email (SMTP/IMAP), DNS, Packet/Circuit Switching. Study the concepts — not just the answers! Good luck! ■